Visão — Plataforma RAG de Documentos

Nov 2, 2026 · @Jorge Nascimento

Visão geral

A plataforma recebe documentos, divide em chunks sem perder contexto, indexa em uma base vetorial e responde perguntas por API, sempre citando documento e versão de origem. Além do RAG, ela gerencia o ciclo de vida dos documentos com versionamento e controle de acesso.

Princípios

- **Simplicidade primeiro:** Postgres com pgvector antes de um banco vetorial dedicado; pipeline próprio em vez de frameworks pesados.
- **Contexto preservado:** chunking orientado à estrutura do documento, nunca só por tamanho fixo.
- **Só a versão vigente responde:** versões antigas não contaminam respostas, mas continuam consultáveis.
- **Segurança na consulta:** cada usuário só recupera chunks de documentos que pode ver.
- **Qualidade medida:** toda mudança de chunking ou modelo passa por um conjunto de avaliação.

Escopo funcional e API

A API cobre três frentes: ingestão assíncrona, consulta com fontes e gestão de versões. O upload responde na hora e o processamento roda em segundo plano.

Endpoint	Função
`POST /documents`	Envia um documento novo (cria a versão 1)
`GET /documents/{id}`	Consulta metadados e status de processamento
`POST /documents/{id}/versions`	Envia uma nova versão do documento
`GET /documents/{id}/versions`	Lista o histórico de versões
`POST /documents/{id}/versions/{v}/activate`	Ativa uma versão (inclusive rollback)

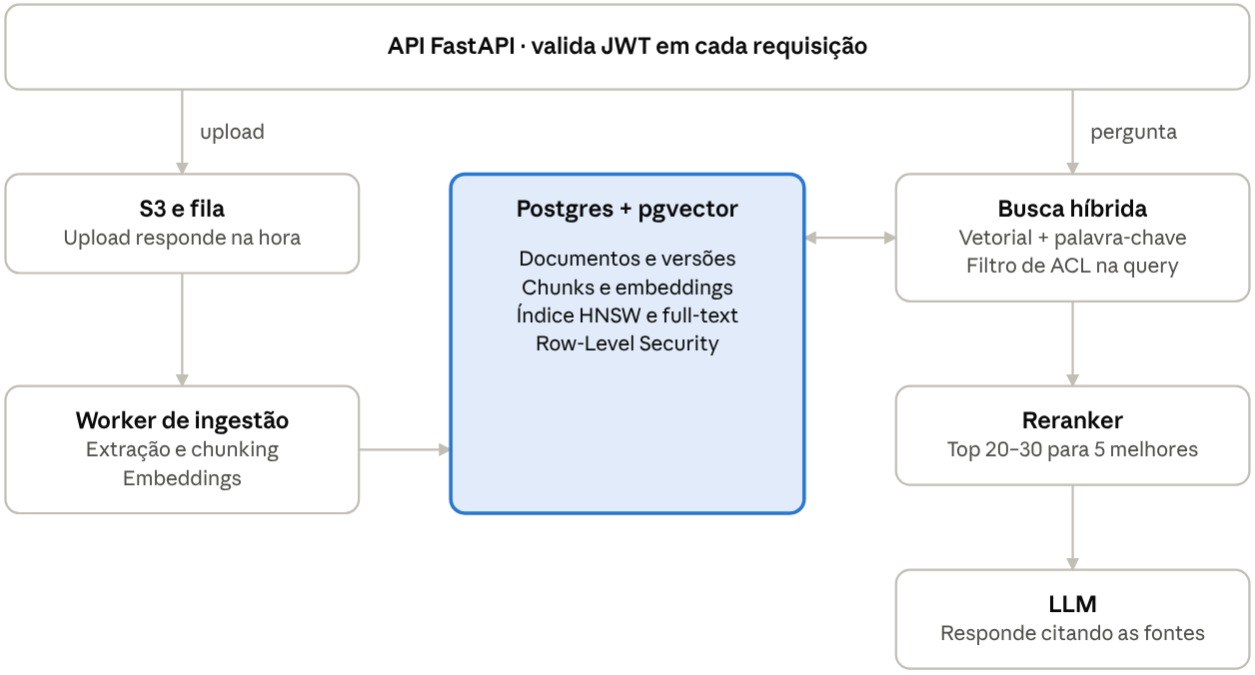
Endpoint	Função
`DELETE /documents/{id}`	Arquiva o documento (soft delete)
`POST /query`	Recebe a pergunta e devolve a resposta com fontes (documento, versão, página, seção)

O `/query` aceita um parâmetro opcional para consultar uma versão histórica específica; por padrão, só a versão ativa é pesquisada.

Arquitetura e stack

A base é Python com FastAPI e Postgres com pgvector, o que evita manter um banco vetorial separado no início. A ingestão roda em workers alimentados por uma fila; a consulta roda direto na API.

Ingestão e consulta se encontram no Postgres com pgvector



arquitetura · fluxo de ingestão e de consulta

À esquerda, a ingestão grava chunks e embeddings no Postgres; à direita, a consulta busca ali, reordena e gera a resposta.

Camada	Escolha	Observação
API	FastAPI	Valida JWT em cada requisição

Camada	Escolha	Observação
Armazenamento de originais	S3 ou MinIO	Um objeto por versão
Fila de ingestão	Celery, RQ ou Arq	Upload não bloqueia
Extração de texto	Docling (PDF estruturado), PyMuPDF, python-docx	Docling preserva títulos e tabelas
Embeddings	BGE-M3 (local) ou API (OpenAI, Voyage)	Modelo fixado desde o início; trocar exige reindexar
Banco vetorial	Postgres + pgvector, índice HNSW	Migrar para Qdrant só com milhões de chunks ou filtros pesados
Busca por palavra-chave	Full-text do Postgres	Base da busca híbrida
Reranker	bge-reranker ou Cohere Rerank	Reordena o top 20–30
Geração	LLM (ex.: Claude)	Responde citando as fontes
Avaliação	RAGAS	Conjunto de 30–50 perguntas

LangChain e LlamaIndex ajudam a prototipar, mas o pipeline é simples o bastante para ser escrito direto, com menos abstração e mais controle.

Estratégia de chunking

Chunks fixos perdem contexto por dois motivos: cortam ideias no meio e deixam cada pedaço sem saber de qual documento ou seção veio. A estratégia combina cinco técnicas, separando o tamanho ideal para *buscar* do tamanho ideal para *responder*.

1. **Corte estrutural.** Dividir pela hierarquia do documento: seção, parágrafo, frase. Só quebrar um parágrafo quando passar do limite (500–800 tokens, sobreposição de 10–15%). Tabelas e listas ficam inteiras.
2. **Prefixo com o caminho.** Antes do embedding, adicionar um cabeçalho como "Contrato X > Cláusula 5 > Rescisão". Resolve chunks ambíguos como "o prazo será de 30 dias".
3. **Contextual Retrieval.** Um LLM barato lê o documento e escreve 1–2 frases situando cada chunk; o texto é colado ao chunk antes do embedding e do índice de palavra-chave. A Anthropic reportou cerca de 49% menos falhas de recuperação, e cerca de 67% com reranker. Prompt caching do documento reduz o custo.
4. **Parent-child (small-to-big).** Indexar chunks pequenos (~200 tokens), precisos na busca, e entregar ao LLM o bloco pai maior (a seção ou ~1000 tokens).

5. **Expansão por vizinhos.** Ao recuperar o chunk N, incluir N−1 e N+1. Substitui o parent-child em documentos sem estrutura (texto corrido, transcrições, OCR) e limita o contexto quando a seção é grande demais.

Regra de recuperação unificada: cada chunk guarda `doc_id`, `section_id` e `position`. Se a seção couber no limite (~1500 tokens), envia-se a seção inteira; se não, a janela de vizinhos. Um único pipeline atende documentos estruturados e não estruturados.

Ordem de adoção: começar pelos itens 1, 2 e 4 (com 5 como fallback), que são baratos e cobrem a maior parte do problema. Adicionar o 3 se a avaliação mostrar chunks ambíguos sendo perdidos. Chunking semântico por similaridade de embeddings fica fora: os ganhos em benchmarks são inconsistentes e o custo é maior.

Recuperação e qualidade

Três práticas concentram o maior ganho de qualidade depois do chunking.

- **Busca híbrida:** combinar busca vetorial com busca por palavra-chave (full-text do Postgres). Ajuda muito com códigos, siglas e nomes próprios.
- **Reranker:** recuperar 20–30 chunks e reordenar com um cross-encoder antes de enviar os 5 melhores ao LLM.
- **Avaliação contínua:** manter 30–50 perguntas com respostas esperadas e medir com RAGAS a cada mudança de chunking, modelo de embedding ou reranker.

Na geração, os chunks entram no prompt delimitados e o LLM é instruído a responder só com base neles, citando documento, versão, página e seção.

Gestão de documentos e versionamento

Cada documento tem várias versões e apenas uma ativa; os chunks pertencem à versão, não ao documento. Isso impede que trechos desatualizados se misturem às respostas.

Tabela	Conteúdo principal
`documents`	Identidade lógica: nome, tipo, dono, tags, ACL, `tenant_id`
`document_versions`	Número da versão, hash SHA-256 do arquivo, caminho no S3, status (`processing` , `active` , `archived`), autor, data
`chunks`	Texto, embedding, `version_id` , `section_id` , `position` , hash do texto, `embedding_model` , metadados de permissão

Regras

1. **Uma versão ativa por documento.** A busca padrão filtra `status = 'active'` ; versões históricas só com parâmetro explícito.
2. **Troca atômica.** A nova versão só vira ativa quando todos os seus chunks estiverem indexados; até lá, a anterior continua respondendo.
3. **Deduplicação por hash.** Reenvio do mesmo arquivo não é reprocessado.
4. **Reaproveitamento de embeddings.** Chunks com hash de texto já existente copiam o embedding em vez de recalcular.
5. **Soft delete.** Excluir arquiva as versões, preservando auditoria e recuperação.
6. **Modelo de embedding registrado.** Permite reindexar em paralelo ao trocar de modelo, sem derrubar a busca.

Um diff de texto entre versões, para revisar antes de ativar, fica para uma fase posterior.

Segurança

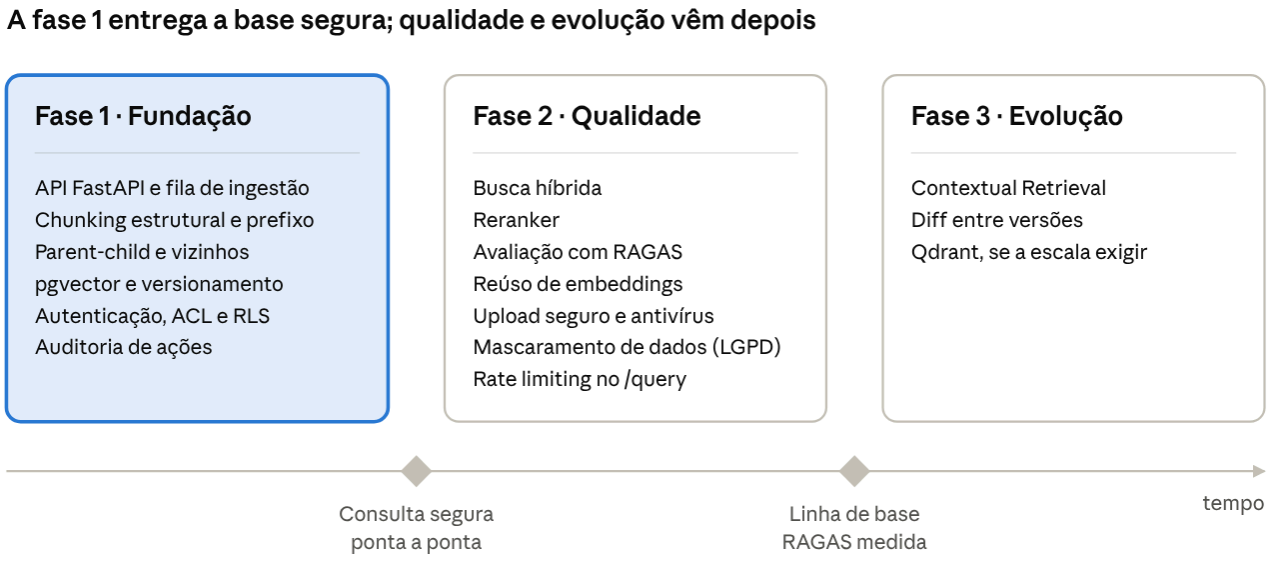
O controle mais importante é garantir que cada usuário só recupere chunks de documentos que pode ver, com o filtro aplicado dentro da própria query vetorial.

Área	Controle
Autenticação	OIDC/JWT com provedor pronto (Keycloak, Auth0, Entra ID, Cognito)
Autorização	ACL por documento (usuários, grupos, `tenant_id`) copiada para os chunks; filtro na query, nunca pós-filtro; Row-Level Security do Postgres como segunda barreira

Área	Controle
Upload	Tipo real validado pelos bytes, limite de tamanho, antivírus (ClamAV), parsing em worker isolado
Prompt injection	Chunks delimitados no prompt e tratados como dados; sem ferramentas com efeito colateral no fluxo de consulta; resposta escapada no front
Dados pessoais (LGPD)	Detecção e mascaramento de CPF, e-mail e telefone na ingestão (Presidio com regras brasileiras); verificar local de processamento e retenção de APIs externas
Infraestrutura	TLS em tudo, criptografia em repouso (S3 e Postgres), segredos em cofre, rate limiting por usuário no `/query`
Auditoria	Registro de upload, ativação, consulta e exclusão, incluindo os chunks usados em cada resposta

Autenticação, autorização com RLS e auditoria são prioridade desde o início: são os controles mais caros de adicionar com o sistema em produção.

Roadmap em fases



roadmap · 3 fases, 2 marcos

A fase 1 concentra o que é caro de adicionar depois (autenticação, ACL com RLS, auditoria e versionamento). A fase 2 só começa com a consulta segura funcionando ponta a ponta, e a fase 3 parte de uma linha de base de métricas para justificar cada evolução.

Questões em aberto

- [ ] Qual o volume esperado de documentos e de chunks no primeiro ano?
- [ ] Embeddings locais (BGE-M3) ou via API?
- [ ] Qual LLM será usado na geração, e onde os dados podem ser processados?
- [ ] A plataforma será multi-empresa (multi-tenant) desde o início?
- [ ] Quais formatos de arquivo entram no escopo inicial?
- [ ] Qual provedor de identidade a organização já usa?